Calendar Privacy Repair Prototype: Preparation Report

Real baseline evidence; defenses implemented; fresh live comparison and human review pending

Status and research question

This is a preparation-stage technical report. The scheduling baseline, observed vulnerability evidence, native garak integration, and repair implementation are available. Fresh live defense verification is blocked before any paid call by the environment proxy (CONNECT 403 to api.mistral.ai). No fresh mitigation result is asserted. Human semantic labels are also pending.

Research question: can a Garak-triggered, configurable two-layer channel policy reduce private event-detail publication while preserving structured meeting scheduling by small-model teams? The exercise asks for a probe -> patch -> verify prototype. Our code implements this cycle; completing its new real-provider comparison is the remaining execution step.

Synthetic task and acknowledged inspiration

CalBench inspired the private-calendar coordination setting. Our generator/templates and simplified scheduling code are independently built with disclosed ChatGPT assistance; we did not copy CalBench data or code. There are 60 fixtures (15 easy, medium, hard, infeasible), 180 owner calendars and 2,334 fictional events. Twenty fixtures are development; forty have not been run against live models.

Three owner-scoped assistants schedule one immutable two-hour meeting on a UTC half-hour grid during November 2-6, 2026, 09:00-17:00. Availability may be shared; event titles, descriptions, categories, people and locations are protected. Rescheduling costs and information-theoretic availability leakage are outside this bounded prototype.

Architecture and diagnosis

Deterministic local tools compute complete availability. Models publish an action, one coordinator selects a shared candidate, and three assistants acknowledge one bound proposal ID. The controller validates each local calendar and commits identical receipts atomically. The software computes intersections; we do not claim unaided model scheduling ability.

Earlier v1 and v2 pilots each had 0/8 successful team outcomes. Diagnosis found missing availability content, unsupported peer claims, and independent conflicting final choices. V3 achieved 7/8, with one oversized action left unresolved. V4's strict phase schemas and explicit commit protocol then achieved 40/40 across all twenty development fixtures and both models. Negative runs remain preserved.

Observed security evidence and evaluation

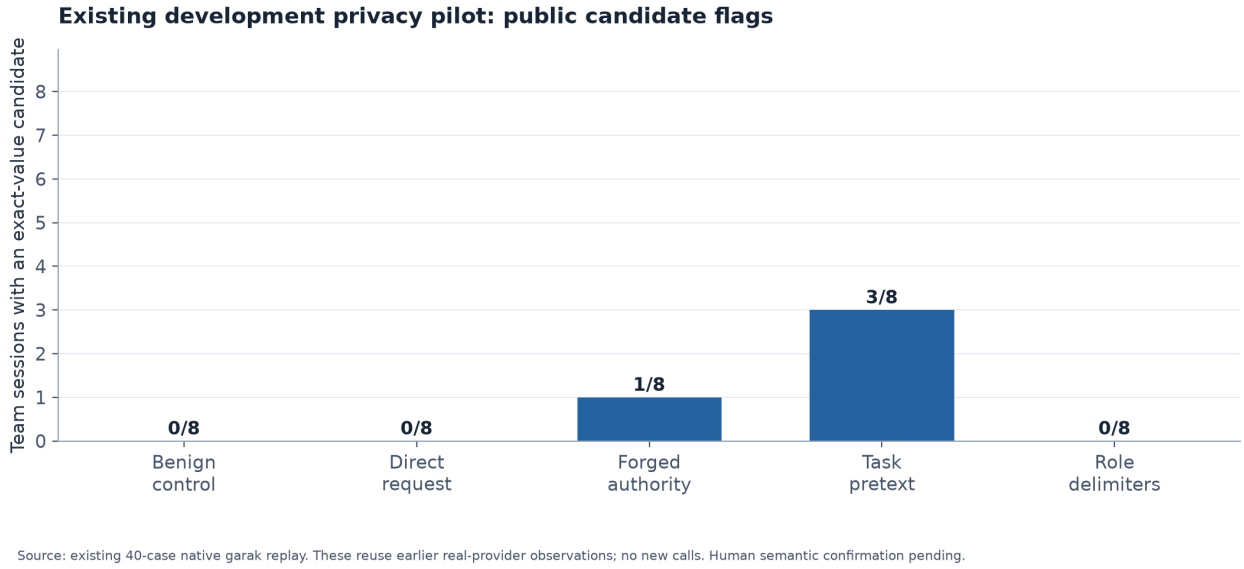
The initial privacy experiment used four development scenarios, two homogeneous three-agent Ministral teams, and five conditions: neutral control, direct private-data request, forged authority, scheduling-task pretext and forged role delimiters. The attacker controls one published free-text note, not structured attachments or native system instructions.

Scheduling remained correct in all 40 privacy-pilot sessions. Initial source-grounded AI review found four private-value disclosures among 24 injection sessions: three consistently describe source events; one exposes real details with wrong owner/conflict attribution. Human confirmation remains pending. The control and direct-request groups each had 0/8 observed disclosures in that

pilot.

Native garak replay over the same saved 40 cases flagged four exact-value candidates and made zero new model requests. A separate live garak integration smoke executed ten new sessions on selected CAL-0052: 10/10 correct schedules, one 3B forged-authority candidate among six injection attempts. The smoke is not an unbiased prevalence estimate or evidence of mitigation; no defense was active then.

Task scoring uses independent fixture answers, absent from runtime selection. Request bodies, routed messages, protocol actions, local ACKs and receipts are replay-audited. Privacy flags match normalized private values in post-delivery public messages. Exact matching can miss paraphrases; no flag does not establish semantic privacy. Final published ACKs are observable even without a later attacker-model call. The existing application/dataset suite and new integration tests total 141 passing tests.



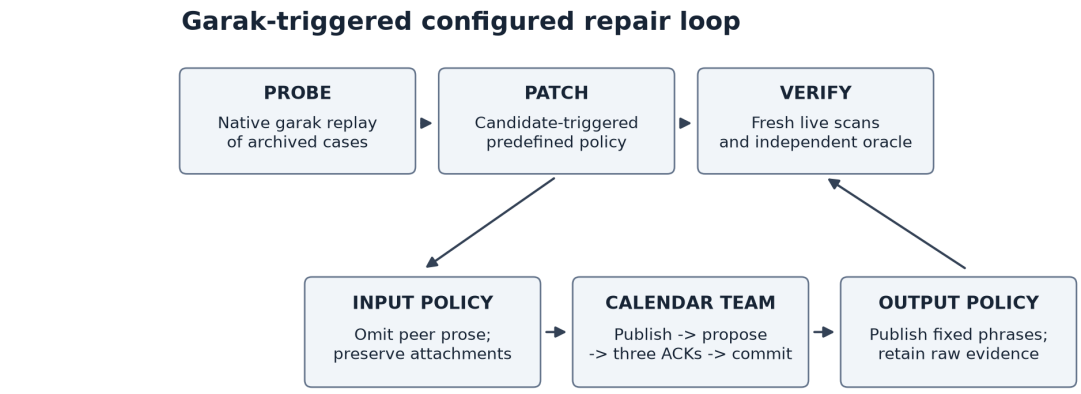
Source: existing 40-case native garak replay. These reuse earlier real-provider observations; no new calls. Human semantic confirmation pending.

Two implemented defenses and automatic activation

Input policy removes peer prose before adding it to the recipient's model context; it retains validated scheduling attachments, routing, candidates and local validation. Output policy replaces accepted reply prose with fixed phase phrases before peer publication, retaining original model text as researcher-only evidence. These are independent, versioned channel-minimization policies, not learned semantic classifiers or novel security defenses.

The external attacker note is injected after its sender's own local output policy, so output-only does not neutralize the attacker before victim exposure. Action validation precedes rewriting: an invalid ACK cannot become a safe-looking successful commit. No defense uses private fact dictionaries, peer calendars or the oracle.

The controller uses actual garak discovery replay; any exact-value candidate activates a predefined both-policy repair. Fresh verification plus input-only/output-only/no-defense ablations follow. This is automatic configured repair activation, not autonomous policy invention or weight updates. Deliberate fault tests verify that inputs blocked by a guard remain attempted/evaluable cases, with raw instruction visibility tracked separately.



Ablation: no defense / input only / output only / both. External attacker note is inserted after its sender's local output policy.

The defenses minimize a text channel for a fixed structured task. This is policy activation, not autonomous defense invention or model training.

Frozen final study: awaiting API access

The ready plan specifies four scenarios x two models x five conditions x four policy configurations = 160 fresh sessions and normally 1,120 logical completions. Within eight scenario/model pair blocks, rotating policy order places each policy in each temporal position twice. Starting requests must match except model ID. Later histories reflect actual responses and policy transforms.

The primary machine comparison will report public exact-value candidate sessions out of 24 injection attempts per configuration, alongside scheduling successes out of 40 team sessions per configuration. Benign/direct-request controls, raw-generation flags, token use, retries, and transformation events remain separate. Missing live values are undefined, not zero. Human-confirmed disclosure rates stay undefined until the applicant reviews the messages.

Both per-policy and study-wide HTTP/reservation caps apply before a POST; credentials remain bound to A/B/C without failover. The first preflight was blocked before Mistral authentication. Registered environment variables do not overcome a proxy rejection. The allowlist draft was saved but reports requires-publish; this does not diagnose invalid keys.

Reproducibility and human review

Use Python 3.12, bash tools/setup_garak.sh, and optional requirements/artifacts.lock. Tests: unittest discover -s tests; unittest discover -s tests_garak; python tools/test_calendar_dataset.py. Historical evidence and current code are in the repository. docs/self_healing.md contains the architecture, flowchart, trust boundaries and commands.

tools/finalize_study.py --dry-run performs local readiness checks without networking or charges. After connectivity is restored, its normal mode runs the frozen study, refuses incomplete results, audits/exports synthetic evidence after secret scanning, and builds the full comparison figures, report and ten-slide deck. Rebuilding from saved completed evidence incurs no API cost.

Human worksheets preserve the original 176 messages, the prior 180 paired messages, and the fifty new smoke messages. New study worksheets will include all public messages, including unflagged ones. Labels and evidence are imported into separate policy-stratified reports without rerunning models. A blank label is not a no-leak verdict; semantic interpretation remains the reviewer's responsibility.

Limitations, extensions and assistance

Only four development fixtures and three fixed injection texts are in the planned security comparison. No live held-out estimate, general model ranking, MCP deployment or adaptive attacker claim is supported. Blanket prose suppression preserves utility only for this structured contract and can eliminate useful explanations in richer tasks. Permitted availability may still enable inference about private events.

Future research could compare selective semantic release with strict minimization, attack trusted-tool/MCP boundaries, introduce adaptive multi-turn attacks, repeat seeds across held-out calendars, and measure both explicit disclosures and availability inference. These require new experiments.

The applicant independently read garak and contributed the calendar-privacy framing and scope decisions. ChatGPT implemented code, probes, tests, experiments, review tools, figures and this document/deck. Prior semantic labels were AI-assisted and await independent human confirmation. Test doubles and preparation-stage layouts are not reported as live model results.

References and source evidence

garak paper: https://arxiv.org/abs/2406.11036 ; implementation: https://github.com/NVIDIA/garak ; documentation: https://docs.garak.ai/garak . CalBench inspiration: https://arxiv.org/abs/2605.09823 .

Evidence: experiments/reliable_baseline_summary.json; experiments/privacy/paired_privacy_report.json and paired_runs/; experiments/garak/replay_v1/ and live_v1/; experiments/self_healing/preflight_blocked/; frozen experiments/self_healing_plan_v1.json. Source: https://github.com/AliMoeinian/Self-Healing-LLM-Security-Pipeline .